University of Calgary
CPSC 433 – Artificial Intelligence

Course Timetabling via Three Search Paradigms

OR-Tree Feasibility, Set-Based Improvement, and AND-Tree Optimization

Formal Search Models and Processes – Proposal

Member 1: Syed Essam Uddin Khawaja 30242943

Member 2: Nova Driscoll 30211940

Member 3: [NAME / UCID]

Member 4: [NAME / UCID]

Member 5: [NAME / UCID]

Date: [SUBMISSION DATE]

Contents

1 Introduction **3**

1.1 Scheduling problem and motivation . . . 3

1.2 Models proposed . . . 3

2 Formal Scheduling Problem **3**

2.1 Components, slots, and domains . . . 4

2.2 Hard constraints . . . 5

2.3 Soft constraints and objective . . . 7

3 Overall Search Architecture **8**

3.1 How the models relate . . . 8

4 Model I: OR-Tree Feasibility Search **8**

4.1 Search model: $A_\vee = (S_\vee, T_\vee)$ . . . 8

4.2 Search process: $P_\vee = (A_\vee, Env_\vee, K_\vee)$ . . . 9

4.3 Search instance: $Inst_\vee = (q_0, G_\vee)$ . . . 9

4.4 Algorithm 1: OR-tree feasibility . . . 10

4.5 Worked OR-tree derivation . . . 10

5 Model II: Set-Based Local Optimization **10**

5.1 Search model: $A_{set} = (S_{set}, T_{set})$ . . . 10

5.2 Search process: $P_{set} = (A_{set}, Env_{set}, K_{set})$ . . . 11

5.3 Search instance: $Inst_{set} = (q_0, G_{set})$ . . . 11

5.4 Algorithm 2: Set-based hill climbing . . . 12

5.5 Worked set-based derivation . . . 12

6 Model III: AND-Tree Optimization **12**

6.1 Motivation and difference from OR-tree . . . 12

6.2 Search model: $A_\wedge = (S_\wedge, T_\wedge)$ . . . 12

6.3 Search process: $P_\wedge = (A_\wedge, Env_\wedge, K_\wedge)$ . . . 13

6.4 Search instance: $Inst_\wedge = (q_0, G_\wedge)$ . . . 14

6.5 Algorithm 3: AND-tree branch-and-bound . . . 14

6.6 Worked AND-tree derivation . . . 14

7 End-to-End Integration and Comparison **15**

7.1 Algorithm 4: Optional combined pipeline 15
7.2 Comparison of the three paradigms 15
7.3 Unified worked example 15

8 Discussion and Limitations **15**

9 Conclusion **16**

A Complete Constraint Formulas **16**

B Additional Derivations and Search Trees **16**

Abstract

[**TO FILL:** Write 150–200 words: problem, all three paradigms, different state representations, search controls, and the benefit/limitation of each. Write this last.]

1 Introduction

1.1 Scheduling problem and motivation

[**TO FILL:** Describe the University of Calgary weekly lecture/tutorial scheduling problem. Distinguish hard constraints from soft penalties.]

We are aiming to solve the problem of creating academic schedules for a semester at the University of Calgary, by making a single weekly schedule (which is then repeated for 13 weeks) containing *Lectures* and *Tutorials* sections (belonging to a course) which fit in weekly room time *Slots* with designated start and end times. There can be multiple *Tutorials* sections which subdivide a particular *Lectures* section and there can be more than one *Lectures* section. Furthermore, there are a number of specified hard and soft constraints on how each lecture/tutorial can fit into the available room time *Slots* (from Monday-Friday). There are two categories of room time *Slots*, one for lectures and the other for tutorials, along with specific "active learning" designation for some rooms, with the number of rooms limited per slot. This problem is similar to what a student would do when creating a weekly class schedule in Schedule Builder, although we are creating the section times which students pick from.

This problem requires a search paradigm, rather than implementing it in traditional computation based programming

The objective is to assign lectures and tutorials to permitted weekly slots so that every hard constraint holds and the soft-constraint penalty is minimized. This proposal compares two ways to improve a valid timetable: OR-tree feasibility followed by set-based local search, or OR-tree feasibility followed by AND-tree branch-and-bound. Section 2 formalizes the scheduling instance once; later sections instantiate the search models and controls without redefining the same constraints.

1.2 Models proposed

We consider three search paradigms:

- **OR-tree**: find one feasible timetable.
- **Set-based**: improve a feasible timetable through local changes.
- **AND-tree**: systematically explore scheduling decisions to seek a globally optimal timetable, using branch-and-bound.

[**TO FILL:** Explain whether the three models will be proposed as independent alternatives, a sequential hybrid, or both. State clearly what the instructor would select to implement.]

2 Formal Scheduling Problem

We use the terminology and symbols of the problem description [2] throughout. Symbols it does not define, such as *overlap*, $D(x)$ and Consistent, are introduced below.

2.1 Components, slots, and domains

Components: *Lectures* is the set of lecture sections and *Tutorials* the set of tutorials and labs; labs (`LAB`) are treated exactly like tutorials (`TUT`). We write $X = \mathit{Lectures} + \mathit{Tutorials}$ for the set of all components to schedule. Each lecture is named *program, course number, section*, for example `CPSC 433 LEC 01`. For $x \in X$ we write $\mathit{course}(x)$ for the pair (program, number), e.g. (`CPSC`, 433).

Which lectures a tutorial belongs to: A tutorial must not clash with its lecture. The tutorial's name tells us which lecture that is, and there are two naming forms:

- `CPSC 433 LEC 01 TUT 02` names a section. It belongs to `CPSC 433 LEC 01` only.
- `CPSC 433 TUT 01` names no section. It is a shared tutorial, open to students of every section, so it belongs to all lectures of `CPSC 433`.

We write $\mathit{Lec}(t)$ for the set of lectures tutorial t belongs to:

$$\mathit{Lec}(t) = \begin{cases} \{\text{the lecture named in } t\} & \text{if } t\text{'s name has a } \texttt{LEC} \text{ part,} \\ \{\, l \in \mathit{Lectures} : \mathit{course}(l) = \mathit{course}(t) \,\} & \text{otherwise (shared tutorial).} \end{cases} \tag{1}$$

For example, if the input has `CPSC 433 LEC 01` and `CPSC 433 LEC 02`, then

$$\mathit{Lec}(\texttt{CPSC 433 LEC 02 TUT 01}) = \{\texttt{CPSC 433 LEC 02}\},$$
$$\mathit{Lec}(\texttt{CPSC 433 TUT 01}) = \{\texttt{CPSC 433 LEC 01}, \texttt{CPSC 433 LEC 02}\}.$$

Weekly slots: The UofC rules link days together. A lecture placed on Monday must also meet Wednesday and Friday at the same time; anything placed on Tuesday must also meet Thursday; a tutorial placed on Monday must also meet Wednesday. So the scheduler never picks single days. It picks a *day pattern and a start time*, and the linked days follow automatically. We call one such choice a *slot*. Because the linking is built into the slot, the day-linking rules always hold and do not appear among the hard constraints in Section 2.2.

Formally, a slot is a triple

$$s = (\mathit{days}(s),\ \mathit{start}(s),\ \mathit{end}(s)), \tag{2}$$

meaning "meet on every day in $\mathit{days}(s)$, from $\mathit{start}(s)$ to $\mathit{end}(s)$." For example, the lecture slot used for a Monday 9:00 lecture is $(\{\text{Mon}, \text{Wed}, \text{Fri}\},\ 9{:}00,\ 10{:}00)$. For short, we name a slot by its day pattern and start time, so this slot is written *MWF 9:00*. The day patterns are written MWF (Mon/Wed/Fri), TT (Tue/Thu), MW (Mon/Wed) and F (Fri only).

Lectures and tutorials use different day patterns; Table 1 lists every slot pattern.

A given input offers only some of these slots, each with its capacities. *Slots* is the set of offered slots. We write Slots_L for the offered lecture slots (top half of Table 1) and Slots_T for the offered tutorial slots (bottom half), so $\mathit{Slots} = \mathit{Slots}_L + \mathit{Slots}_T$. Lectures may only use Slots_L and tutorials only Slots_T.

Time overlap: Since lecture and tutorial slots follow different grids, we compare actual meeting times. Two slots s and s' *overlap* when they meet on a common day and their time ranges intersect:

$$\mathit{overlap}(s, s') \iff \mathit{days}(s) \cap \mathit{days}(s') \neq \varnothing \ \wedge\ \mathit{start}(s) < \mathit{end}(s') \ \wedge\ \mathit{start}(s') < \mathit{end}(s). \tag{3}$$

For example, lecture slot TT 9:30 (9:30-11:00) overlaps tutorial slots TT 9:00 and TT 10:00.

Table 1: Slot patterns. Each row is one group of slots; h is the start time.

Used by	Written as	Meets on	Length	Start times	Count
Lectures	MWF h	Mon, Wed, Fri	1 h	8:00, 9:00, …, 20:00	13
Lectures	TT h	Tue, Thu	1.5 h	8:00, 9:30, …, 18:30	8
Tutorials	MW h	Mon, Wed	1 h	8:00, 9:00, …, 20:00	13
Tutorials	TT h	Tue, Thu	1 h	8:00, 9:00, …, 20:00	13
Tutorials	F h	Fri only	2 h	8:00, 10:00, …, 18:00	6

Two lectures, or two tutorials, can only clash if they are in the same slot, because slots of the same kind either fall on different days or follow each other without overlapping:

$$s, s' \in \mathit{Slots}_L \;\text{ or }\; s, s' \in \mathit{Slots}_T \implies (\mathit{overlap}(s, s') \iff s = s'). \tag{4}$$

Domains: The *domain* $D(x)$ is the set of slots component x may use: the slots of its kind (Slots_L or Slots_T), minus any slot ruled out by a hard constraint on x alone (C_5, C_8 to C_{10} and C_{12} in Section 2.2).

Assignments: A complete timetable is a function

$$\mathit{assign} : \mathit{Lectures} + \mathit{Tutorials} \to \mathit{Slots}, \qquad \mathit{assign}(x) \in D(x) \text{ for every } x \in X. \tag{5}$$

While a timetable is being built, only some components have a slot yet. Such a *partial assignment* *assign* is defined only on the components $\mathrm{dom}(\mathit{assign}) \subseteq X$ placed so far. (This is different from the input preassignment *partassign* in Section 2.2.)

Slot counts: The capacity constraints (C_1 to C_4) need to know how full each slot is. For an assignment *assign* and a slot s, $n_L(s)$ is the number of lectures *assign* puts in s and $n_T(s)$ the number of tutorials. $n_{L,AL}(s)$ and $n_{T,AL}(s)$ count only the ones with an *activelearning* statement. Only placed components are counted, so the same counts work for partial assignments:

$$\begin{aligned}
n_L(s) &= |\{l \in \mathit{Lectures} \cap \mathrm{dom}(\mathit{assign}) : \mathit{assign}(l) = s\}|,\\
n_T(s) &= |\{t \in \mathit{Tutorials} \cap \mathrm{dom}(\mathit{assign}) : \mathit{assign}(t) = s\}|,\\
n_{L,AL}(s) &= |\{l \in \mathit{Lectures} \cap \mathrm{dom}(\mathit{assign}) : \mathit{activelearning}(l),\ \mathit{assign}(l) = s\}|,\\
n_{T,AL}(s) &= |\{t \in \mathit{Tutorials} \cap \mathrm{dom}(\mathit{assign}) : \mathit{activelearning}(t),\ \mathit{assign}(t) = s\}|.
\end{aligned}$$

For example, if *assign* puts `CPSC 433 LEC 01` and `DATA 201 LEC 01` in MWF 9:00 and only `DATA 201 LEC 01` needs active learning, then $n_L(\text{MWF 9:00}) = 2$ and $n_{L,AL}(\text{MWF 9:00}) = 1$. C_1 then checks $2 \le \mathit{lecturemax}(\text{MWF 9:00})$ and C_3 checks $1 \le \mathit{ALlecturemax}(\text{MWF 9:00})$.

2.2 Hard constraints

Hard constraints are the rules a valid timetable cannot break. Besides the sets defined above, they use these inputs:

- lecture capacities $\mathit{lecturemax}(s)$ and $\mathit{ALlecturemax}(s)$ for each $s \in \mathit{Slots}_L$;
- tutorial capacities $\mathit{tutorialmax}(s)$ and $\mathit{ALtutorialmax}(s)$ for each $s \in \mathit{Slots}_T$;

- the *notcompatible*(x, y) and *unwanted*(x, s) statements;
- the preassignment *partassign* : *Lectures* + *Tutorials* $\to$ *Slots* + {\$}, where \$ means no preassignment;
- the *activelearning*(x) statements.

We write *ALmax*(s) for *ALlecturemax*(s) if $s \in Slots_L$ and *ALtutorialmax*(s) if $s \in Slots_T$. Each constraint only looks at components that have been placed, so it can also be checked on a partial assignment.

$$
\begin{aligned}
C_1 &: \forall s \in Slots_L : \ n_L(s) \le lecturemax(s) && \text{(lecture capacity)} && (6)\\
C_2 &: \forall s \in Slots_T : \ n_T(s) \le tutorialmax(s) && \text{(tutorial capacity)} && (7)\\
C_3 &: \forall s \in Slots_L : \ n_{L,AL}(s) \le ALlecturemax(s) && \text{(AL lecture capacity)} && (8)\\
C_4 &: \forall s \in Slots_T : \ n_{T,AL}(s) \le ALtutorialmax(s) && \text{(AL tutorial capacity)} && (9)\\
C_5 &: \forall x \text{ with } activelearning(x) : \ ALmax(assign(x)) > 0 && \text{(AL placement)} && (10)\\
C_6 &: \forall t \in Tutorials, \ \forall l \in Lec(t) : \ \neg\, overlap(assign(l), assign(t)) && \text{(lecture/tutorial)} && (11)\\
C_7 &: \forall\, notcompatible(x, y) : \ \neg\, overlap(assign(x), assign(y)) && \text{(not compatible)} && (12)\\
C_8 &: \forall x \text{ with } partassign(x) \neq \$: \ assign(x) = partassign(x) && \text{(preassignment)} && (13)\\
C_9 &: \forall\, unwanted(x, s) : \ assign(x) \neq s && \text{(unwanted)} && (14)\\
C_{10} &: \forall x \in X_9 : \ start(assign(x)) \ge 18{:}00 && \text{(LEC 9 evening)} && (15)\\
C_{11} &: \forall l \neq l' \in L_5 : \ assign(l) \neq assign(l') && \text{(5XX non-overlap)} && (16)\\
C_{12} &: \forall l \in Lectures : \ assign(l) \neq \text{TT } 11{:}00 && \text{(TT 11:00 meeting)} && (17)\\
C_{13} &: \text{special CPSC 851/913 bookings (below)}
\end{aligned}
$$

X_9 is the set of `LEC 9` sections and their tutorials (e.g. `CPSC 433 LEC 90 TUT 01`), and L_5 is the set of 5XX lectures across all programs. The problem description writes C_6 as $assign(l) \neq assign(t)$; since lecture and tutorial slots are never equal, we read it as "must not overlap".

Special bookings (C_{13}): If the input has a CPSC 351 lecture, then for every CPSC 851 component t we set $partassign(t) = \text{TT } 18{:}00$ and add $notcompatible(t, y)$ for every CPSC 351 lecture and tutorial y. If the input lists no CPSC 851 component, we add `CPSC 851 TUT 01`. CPSC 913 is handled the same way with CPSC 413. C_{13} is then checked by C_7 and C_8. We assume these bookings use rooms reserved for them: they are not counted in $n_T(s)$ or $n_{T,AL}(s)$, so they never use up *tutorialmax*(TT 18:00). If the input does not offer TT 18:00 as a tutorial slot, we add it to $Slots_T$ with *tutorialmax*, *ALtutorialmax* and *tutorialmin* all 0, so only these bookings can use it.

Validity: A complete timetable *assign* is valid if it breaks no hard constraint:

$$\boxed{\text{Constr}(assign) = \bigwedge_{i=1}^{13} C_i(assign).} \tag{18}$$

A partial assignment is *consistent* if the components placed so far break no constraint:

$$\text{Consistent}(assign) \iff \text{every } C_i \text{ holds on dom}(assign). \tag{19}$$

Once a partial assignment breaks a rule, placing more components cannot fix it: a full slot stays full and a clash stays a clash. So every model can discard a partial assignment as soon as Consistent fails.

2.3 Soft constraints and objective

Soft constraints may be broken, but each violation adds penalty points. They use these inputs:

- minimums $\mathit{lecturemin}(s)$ for $s \in \mathit{Slots}_L$ and $\mathit{tutorialmin}(s)$ for $s \in \mathit{Slots}_T$;
- $\mathit{preference} : (\mathit{Lectures} + \mathit{Tutorials}) \times \mathit{Slots} \to \mathbb{N}$ (0 where none is given);
- the $\mathit{pair}(x, y)$ statements;
- penalties $\mathit{pen}_{\mathit{lecturemin}}$, $\mathit{pen}_{\mathit{tutorialmin}}$, $\mathit{pen}_{\mathit{notpaired}}$, $\mathit{pen}_{\mathit{section}} \in \mathbb{N}$.

There are five penalties:

Too few lectures: If a lecture slot holds fewer lectures than its minimum, each missing lecture costs $\mathit{pen}_{\mathit{lecturemin}}$. A slot with minimum 3 that holds 1 lecture costs $2 \cdot \mathit{pen}_{\mathit{lecturemin}}$; a slot at or above its minimum costs nothing.

$$E_{L\,\min}(\mathit{assign}) = \mathit{pen}_{\mathit{lecturemin}} \cdot \sum_{s \in \mathit{Slots}_L} \max\bigl(0,\ \mathit{lecturemin}(s) - n_L(s)\bigr) \tag{20}$$

Too few tutorials: The same for tutorial slots.

$$E_{T\,\min}(\mathit{assign}) = \mathit{pen}_{\mathit{tutorialmin}} \cdot \sum_{s \in \mathit{Slots}_T} \max\bigl(0,\ \mathit{tutorialmin}(s) - n_T(s)\bigr) \tag{21}$$

Unmet preferences: If a component does not get a slot it has a preference for, that preference's value is added.

$$E_{\text{pref}}(\mathit{assign}) = \sum_{x \in X} \sum_{s \in \mathit{Slots},\ s \neq \mathit{assign}(x)} \mathit{preference}(x, s) \tag{22}$$

Unmet pairs: Each $\mathit{pair}(x, y)$ statement whose components are not at the same time (defined below) costs $\mathit{pen}_{\mathit{notpaired}}$.

$$E_{\text{pair}}(\mathit{assign}) = \mathit{pen}_{\mathit{notpaired}} \cdot \bigl|\{\, \mathit{pair}(x, y) : \neg\, \mathit{sameTime}(\mathit{assign}(x), \mathit{assign}(y)) \,\}\bigr| \tag{23}$$

Sections sharing a slot: Each pair of lecture sections of the same course placed in the same slot costs $\mathit{pen}_{\mathit{section}}$.

$$\begin{aligned} E_{\text{section}}(\mathit{assign}) = \mathit{pen}_{\mathit{section}} \cdot \bigl|\{\, \{l, l'\} \subseteq \mathit{Lectures} : l \neq l',\ \mathit{course}(l) = \mathit{course}(l'),\\ \mathit{assign}(l) = \mathit{assign}(l') \,\}\bigr| \end{aligned} \tag{24}$$

Same time: For two lectures, or two tutorials, "same time" means the same slot. A lecture and a tutorial never share a slot, so we count them as at the same time if they share a day and start time:

$$\begin{aligned} &\text{same kind:} && \mathit{sameTime}(s, s') \iff s = s', \\ &\text{lecture vs. tutorial:} && \mathit{sameTime}(s, s') \iff \mathit{days}(s) \cap \mathit{days}(s') \neq \varnothing \ \wedge\ \mathit{start}(s) = \mathit{start}(s'). \end{aligned} \tag{25}$$

For example, lecture MWF 8:00 and tutorial MW 8:00 are at the same time; lecture TT 9:30 and tutorial TT 9:00 are not.

Objective: Eval is the total penalty,

$$\boxed{\begin{aligned}\text{Eval}(\mathit{assign}) = E_{L\,\min}(\mathit{assign}) + E_{T\,\min}(\mathit{assign}) + E_{\text{pref}}(\mathit{assign}) \\ + E_{\text{pair}}(\mathit{assign}) + E_{\text{section}}(\mathit{assign}),\end{aligned}} \tag{26}$$

and the problem is

$$\boxed{\min_{\mathit{assign}} \text{Eval}(\mathit{assign}) \quad \text{subject to} \quad \text{Constr}(\mathit{assign}) = \mathsf{true}.} \tag{27}$$

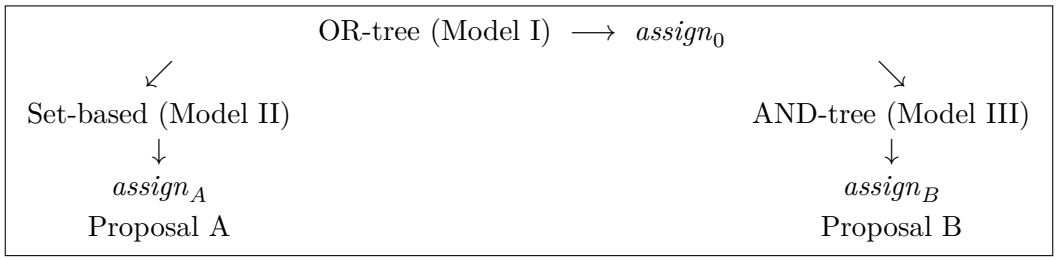
Every penalty is 0 or more, so a valid timetable with $\text{Eval}(\mathit{assign}) = 0$ is optimal.

3 Overall Search Architecture

3.1 How the models relate

We propose two complete search systems that share the same first stage:

- **Proposal A:** OR-tree for an initial valid timetable (Section 4) → set-based improvement (Section 5).
- **Proposal B:** OR-tree for an initial valid timetable (Section 4) → AND-tree optimization (Section 6).



(28)

All three models use the problem definition of Section 2: the components X, the domains $D(x)$, the overlap test (3), the checker Consistent (19) and Eval.

Shared first stage: Both second stages need a valid timetable to start from. The OR-tree (Section 4) finds one, assign_0, or reports that no valid timetable exists. It only looks for feasibility and ignores Eval.

How the stages connect: Both second stages begin with assign_0 as their best timetable and only replace it with a strictly better valid one, so

$$\text{Eval}(\mathit{assign}_A) \le \text{Eval}(\mathit{assign}_0) \quad \text{and} \quad \text{Eval}(\mathit{assign}_B) \le \text{Eval}(\mathit{assign}_0).$$

4 Model I: OR-Tree Feasibility Search

4.1 Search model: $A_\vee = (S_\vee, T_\vee)$

Problem descriptions.

$$\boxed{\mathit{Prob}_\vee = \{(a, U) : a \text{ is partial},\ U = X \setminus \text{dom}(a)\}.} \tag{29}$$

Here a is the current partial timetable, and U contains unscheduled components. A node has the lecture's form

$$((a, U), sol), \qquad sol \in \{\mathsf{yes}, ?, \mathsf{no}\}. \tag{30}$$

Alternatives. Select a component $x \in U$ and define

$$D_a(x) = \{s \in D(x) : \text{Consistent}(a \cup \{x \mapsto s\})\}. \tag{31}$$

For every $s_i \in D_a(x)$, let

$$pr_i = (a \cup \{x \mapsto s_i\}, U \setminus \{x\}). \tag{32}$$

Then the application-specific alternative relation is

$$\boxed{Altern(pr, pr_1, \ldots, pr_r), \qquad r = |D_a(x)|.} \tag{33}$$

States and transitions. Following Lecture 11,

$$S_\vee \subseteq Otree, \qquad T_\vee = \{(q, q') : Erw_\vee(q, q') \text{ or } Erw^*_\vee(q, q')\}. \tag{34}$$

The extension relation marks a solved leaf yes, marks an unsolvable leaf no, or expands a leaf using *Altern*. Extensions can act on leaves inside a larger tree.

[**TO FILL:** Write the four formal $Erw_\vee$ cases from Lecture 11, using the definitions above.]

4.2 Search process: $P_\vee = (A_\vee, Env_\vee, K_\vee)$

[**TO FILL:** Define $Env_\vee$, leaf selection, and slot ordering. Proposed $K_\vee$: depth-first unfinished-leaf selection and MRV component selection. No forward checking is necessary for correctness.]

The MRV component selection is

$$\boxed{x^* = \arg\min_{x \in U} |D_a(x)|.} \tag{35}$$

4.3 Search instance: $Inst_\vee = (q_0, G_\vee)$

Start with

$$a_0 = \varnothing, \quad U_0 = X, \quad q_0 = ((a_0, U_0), ?). \tag{36}$$

A leaf is solved if $U = \varnothing$ and Const(a) is true. The search halts when a solved branch exists or all alternatives have failed; these are distinct outcomes.

[**TO FILL:** Define $G_\vee$ recursively in terms of the tree and distinguish success from exhaustive failure.]

4.4 Algorithm 1: OR-tree feasibility

Algorithm 1 OR-Tree Feasibility Search (operational outline)

Require: Components X, domains D, partial checker Consistent
Ensure: Complete valid timetable, or FAILURE

Initialize tree $q \leftarrow ((\varnothing, X), ?)$
while an unresolved leaf exists **do**
  $v \leftarrow$ deepest unresolved leaf of q
  $(a, U) \leftarrow$ problem description at v
  if $U = \varnothing$ **then**
    if Const(a) **then**
      Mark v as yes; **return** a
    else
      Mark v as no;
      continue
    end if
  end if
  $x \leftarrow \arg\min_{y \in U} |D_a(y)|$ ▷ MRV
  $C \leftarrow D_a(x)$
  if $C = \varnothing$ **then**
    Mark v as no
  else
    for all $s \in C$ in deterministic slot order **do**
      Add child $((a \cup \{x \mapsto s\}, U \setminus \{x\}), ?)$ to v
    end for
  end if
end while
return FAILURE

4.5 Worked OR-tree derivation

[**TO FILL:** Starting with $a_0 = \varnothing$ and $U_0 = X$, show every intermediate (a_i, U_i); draw an OR-tree with at least one failing alternative and one solved leaf. Explain why the search stops on its first successful path.]

5 Model II: Set-Based Local Optimization

5.1 Search model: $A_{set} = (S_{set}, T_{set})$

Facts.

$$\boxed{F = \{\text{Assigned}(x, s) : x \in X,\ s \in D(x)\}.} \tag{37}$$

The fact Assigned(x, s) means that component x occupies weekly slot s.

States.

$$\boxed{S_{set} = \{q \subseteq F : q \text{ encodes one complete valid timetable}\} \subseteq 2^F.} \tag{38}$$

For any $q \in S_{set}$, let a_q be the complete timetable that its facts represent.

Extensions.

$$\begin{aligned} &\text{Move: } \{\text{Assigned}(x, s_1)\} \longrightarrow \{\text{Assigned}(x, s_2)\}; \\ &\text{Swap: } \{\text{Assigned}(x, s_1), \text{Assigned}(y, s_2)\} \\ &\qquad\qquad \longrightarrow \{\text{Assigned}(x, s_2), \text{Assigned}(y, s_1)\}. \end{aligned} \tag{39}$$

Here x, y are distinct components and s_1, s_2 are their current slots. The rules are applicable only if domains, preassignments, and all hard constraints are respected in the resulting timetable.

$$\boxed{q' = (q \setminus A) \cup B, \qquad A \to B \in EXT.} \tag{40}$$

$$T_{set} = \{(q, q') : A \to B \in EXT,\ A \subseteq q,\ q' = (q \setminus A) \cup B,\ q' \in S_{set}\}. \tag{41}$$

5.2 Search process: $P_{set} = (A_{set}, Env_{set}, K_{set})$

The proposed greedy weight function is

$$\boxed{f_{weight}(A, B, e) = \text{Eval}(a_{(q \setminus A) \cup B}),} \tag{42}$$

where the environment e provides the current timetable q and other problem information. The function f_{select} breaks ties between equally weighted extensions.

[**TO FILL:** Define Env_{set}, f_{select}, the strict-improvement condition, and the neighborhood $N(q)$. Explain the possibility of getting stuck at a local optimum.]

5.3 Search instance: $Inst_{set} = (q_0, G_{set})$

The OR-tree produces a_{feasible}. The set-based starting state is

$$\boxed{q_0 = \{\text{Assigned}(x, a_{\text{feasible}}(x)) : x \in X\}.} \tag{43}$$

[**TO FILL:** Define a practical goal/stopping condition: score zero, no improving neighbor, or a specified search budget.]

5.4 Algorithm 2: Set-based hill climbing

Algorithm 2 Set-Based Optimization

Require: Feasible timetable a_{feasible}, EXT, Eval
Ensure: Improved valid timetable

$q \leftarrow \{\text{Assigned}(x, a_{\text{feasible}}(x)) : x \in X\}$
while $\text{Eval}(a_q) > 0$ **do**
  $N \leftarrow \{q' : (q, q') \in T_{set},\ q' \neq q\}$
  if $N = \varnothing$ **then**
    return a_q
  end if
  $m \leftarrow \min_{q' \in N} \text{Eval}(a_{q'})$
  if $m \geq \text{Eval}(a_q)$ **then**
    return a_q ▷ Local optimum
  end if
  $q \leftarrow f_{select}(\{q' \in N : \text{Eval}(a_{q'}) = m\})$
end while
return a_q

5.5 Worked set-based derivation

[**TO FILL:** Represent the OR-tree solution as facts. Show Move and optionally Swap using $q' = (q \setminus A) \cup B$. Give before/after evaluation values and verify $\text{Const}(a_{q'})$.]

6 Model III: AND-Tree Optimization

6.1 Motivation and difference from OR-tree

An OR-tree requires *one* successful alternative. An AND-tree requires the chosen division's subproblems to *all* be solved and mutually compatible. For optimization, different divisions must also be compared to find the best result. The AND-tree may backtrack to another division if the current one fails or cannot beat an existing solution.

[**TO FILL:** Explain why AND-tree plus branch-and-bound is a reasonable alternative to local set-based optimization. Cite Lecture 09.]

6.2 Search model: $A_\wedge = (S_\wedge, T_\wedge)$

Problem descriptions. We propose two kinds of problem description:

$$\boxed{Prob_\wedge = \{\mathsf{Schedule}(a, U)\} \ \cup \ \{\mathsf{Verify}(a, x, s)\}.} \tag{44}$$

Here $\mathsf{Schedule}(a, U)$ asks for a completion of partial assignment a on the remaining components U. The problem $\mathsf{Verify}(a, x, s)$ asks whether putting component x in slot s is consistent with the current timetable a.

Division relation. Select $x \in U$ and a candidate $s \in D(x)$. One possible division is

$$\boxed{\begin{aligned} &Div(\mathsf{Schedule}(a, U),\\ &\quad \mathsf{Verify}(a, x, s),\\ &\quad \mathsf{Schedule}(a \cup \{x \mapsto s\}, U \setminus \{x\})). \end{aligned}} \tag{45}$$

The division has *two necessary subproblems*: check the new assignment, and schedule everything remaining with the new assignment fixed. A different candidate s produces a different possible *Div* choice. Thus this is an AND division, not an OR-tree *Altern* list.

Solved / failed subproblems.

$$\mathsf{Verify}(a, x, s) \text{ is solved} \iff \text{Consistent}(a \cup \{x \mapsto s\}), \tag{46}$$

$$\mathsf{Schedule}(a, \varnothing) \text{ is solved} \iff \text{Const}(a) = \mathsf{true}. \tag{47}$$

If verification fails, that division cannot succeed. The controller must try a different division by the lecture's AND-tree backtracking mechanism.

States and transitions. Lecture 09 defines

$$\boxed{S_\wedge \subseteq \mathit{Atree}, \quad \mathit{sol} \in \{\mathsf{yes}, ?\}.} \tag{48}$$

An AND-tree state is an entire tree of subproblems, not a single partial assignment. Its transitions follow the lecture's expansion and contraction relations:

$$\boxed{T_\wedge = \{(q, q') : \mathit{Erw}_\wedge(q, q') \text{ or } \mathit{Erw}^*_\wedge(q, q')\}.} \tag{49}$$

[**TO FILL:** State the formal AND-tree $\mathit{Erw}_\wedge$ and $\mathit{Erw}^*_\wedge$ rules from Lecture 09. Explain why AND-trees use yes and ? rather than an explicit no node status, and how contraction/backtracking handles failed divisions.]

6.3 Search process: $P_\wedge = (A_\wedge, \mathit{Env}_\wedge, K_\wedge)$

We propose MRV for selecting the next component and branch-and-bound for choosing/pruning divisions. The environment records the best complete timetable found so far, a_{Best}, and its score

$$\boxed{B = \text{Eval}(a_{\mathsf{Best}}).} \tag{50}$$

The valid timetable returned by OR-tree (or improved by set-based search) provides the initial incumbent a_{Best}. For a partial timetable a, let $\mathsf{LB}(a, U)$ be a sound lower bound on the penalty of *every complete valid extension* of a. Then

$$\boxed{\mathsf{LB}(a, U) \geq B \quad \Longrightarrow \quad \text{prune this division.}} \tag{51}$$

The lower bound must satisfy

$$\boxed{\mathsf{LB}(a, U) \leq \text{Eval}(\widehat{a})} \tag{52}$$

for every feasible completion $\widehat{a}$ of a.

[**TO FILL:** Define a concrete admissible lower bound. A safe initial bound is the sum of nonnegative preference and pair/section penalties already made unavoidable by assigned components; omit uncertain minimum-occupancy penalties. Explain how f_{leaf} and f_{trans} select a leaf and a division.]

6.4 Search instance: $Inst_\wedge = (q_0, G_\wedge)$

Begin the AND-tree optimization from the original problem, but pass the best known complete timetable as a bound:

$$\boxed{q_0 = (\mathsf{Schedule}(\varnothing, X), ?).} \tag{53}$$

[**TO FILL:** Define $G_\wedge$ using Lecture 09: root solved, all subproblems solved compatibly, or every applicable division/backtracking option exhausted. Specify separately what certifies optimality: all better candidates have been examined or soundly pruned.]

6.5 Algorithm 3: AND-tree branch-and-bound

Algorithm 3 AND-Tree Optimization (control implementing *Div*)

Require: Original CSP, a complete valid incumbent a_{Best}
Ensure: Best valid schedule found; optimal if exhaustive

$B \leftarrow \mathrm{Eval}(a_{\mathsf{Best}})$
procedure OPTIMIZE(a, U)
  if $\mathsf{LB}(a, U) \geq B$ **then**
    return ▷ Soundly pruned division
  end if
  if $U = \varnothing$ **then**
    if $\mathrm{Const}(a)$ **and** $\mathrm{Eval}(a) < B$ **then**
      $a_{\mathsf{Best}} \leftarrow a$; $B \leftarrow \mathrm{Eval}(a)$
    end if
    return
  end if
  $x \leftarrow \arg\min_{y \in U} |D_a(y)|$ ▷ MRV
  for all $s \in D(x)$ **do**
    Consider division $Div(\mathsf{Schedule}(a, U),$
      $\mathsf{Verify}(a, x, s), \mathsf{Schedule}(a \cup \{x \mapsto s\}, U \setminus \{x\}))$
    if $\mathrm{Consistent}(a \cup \{x \mapsto s\})$ **then** ▷ Verify child
      OPTIMIZE($a \cup \{x \mapsto s\}, U \setminus \{x\}$) ▷ Remaining child
    end if
    ▷ If needed, try alternate *Div* (AND-tree backtracking)
  end for
end procedure
OPTIMIZE($\varnothing, X$)
return a_{Best} and completion/optimality status

This is operational pseudocode for the branch-and-bound control. The formal model remains an AND-tree with *Div* and $Erw^*_\wedge$; the recursive procedure implements selecting divisions and exploring them. It is not an OR-tree *Altern* relation.

6.6 Worked AND-tree derivation

[**TO FILL:** Using the same small timetable instance, show at least two possible *Div* choices for LEC 02: MWF 9–10 and MWF 10–11. Expand one AND division into its Verify and Schedule children.

Show how the incumbent penalty is updated or how a division is pruned. If the starting incumbent has score zero and penalties are nonnegative, explain why that already certifies global optimality.]

7 End-to-End Integration and Comparison

7.1 Algorithm 4: Optional combined pipeline

Algorithm 4 Combined Three-Paradigm Pipeline

Require: Scheduling instance I
Ensure: Best valid timetable found, or FAILURE
1: $a_0 \leftarrow \text{ORFeasibility}(I)$
2: **if** $a_0 = \text{FAILURE}$ **then**
3: **return** FAILURE
4: **end if**
5: $a_1 \leftarrow \text{SetBasedImprove}(I, a_0)$
6: $a_2 \leftarrow \text{ANDOptimize}(I, a_1)$
7: **return** a_2 ▷ Optimal only if AND stage completed

7.2 Comparison of the three paradigms

Table 2: Summary of proposed models

	OR-tree	Set-based	AND-tree
Purpose	Feasibility	Local improvement	Exact optimization
State	OR-tree	Assignment fact set	AND-tree
Primitive	*Altern*	*EXT*	*Div*
Control	DFS + MRV	Hill climbing	Branch-and-bound
Optimal?	Not targeted	Not guaranteed	If complete

[**TO FILL:** Explain each model's pros and cons: search memory, runtime, completeness, and solution quality. Avoid claiming an AND-tree guarantees optimality if the search budget expires.]

7.3 Unified worked example

[**TO FILL:** Use one small course-scheduling instance across all three models. Show the OR-tree's first feasible score, set-based's improved score, and AND-tree's exploration/proof of optimality. The same input and penalty parameters must be used in all three worked examples.]

8 Discussion and Limitations

[**TO FILL:** Explain why three models may be informative but more complex than required. Discuss infeasible inputs, local minima, sound bounding, valid outputs on timeout, and algorithm selection for large instances.]

The shared OR-tree stage may exhaust all alternatives without finding a feasible timetable. If it is interrupted before that determination, infeasibility has not been established. Set-based hill climbing can reach local minima because it only accepts improving Moves or Swaps. AND-tree branch-and-bound preserves a complete feasible incumbent while exploring the full assignment space, but its worst-case work is exponential; only a completed search or a zero lower bound with zero penalty establishes global optimality. For larger instances, a time budget may be needed, and the returned timetable should be labelled feasible rather than optimal when the search is interrupted.

9 Conclusion

[**TO FILL:** Summarize the three models, their distinct state data structures, and why the proposed architecture is appropriate for the assignment. Do not claim experimentally verified performance before testing.]

We proposed two complete timetabling pipelines sharing an OR-tree feasibility stage. The set-based branch offers simple, feasibility-preserving local improvement, whereas AND-tree branch-and-bound searches across alternative assignments and can prove an optimum when completed. Both use the same hard-constraint checker and evaluation objective. The proposed models and worked derivations specify how each search process can be instantiated for the course timetabling problem.

References

[1] CPSC 433. *Project Description*, Version 1, University of Calgary, Fall 2025.

[2] CPSC 433. *Project Problem Description*, Version 1, University of Calgary, Fall 2025.

[3] CPSC 433. *Lecture 05: Search*, University of Calgary.

[4] CPSC 433. *Lecture 06: Set-Based Search*, University of Calgary.

[5] CPSC 433. *Lecture 09: And-Tree-Based Search*, University of Calgary.

[6] CPSC 433. *Lecture 11: Or-Tree-Based Search*, University of Calgary.

[7] CPSC 433. *Lecture 14: Search Controls*, University of Calgary.

A Complete Constraint Formulas

[**TO FILL:** Optional: extended detailed formalization of all hard and soft constraints.]

B Additional Derivations and Search Trees

[**TO FILL:** Optional: extra OR-tree, AND-tree, and set-based state traces.]